

Task 10: Read back from on start-up.

- Read your saved records back into the program when the application boots up, use Java's built-in File and Scanner tools.
- Instead of jumping straight into the interactive loop, the program will look for report_cards.txt the exact moment it starts. If the file exists, the program should open it, read it line-by-line, and print the historical records directly to the screen before presenting the registration menu.

What We Are Adding in This Step:

1. **New Package Import:** We add import java.io.File; to manage file path lookups on your operating system.
2. **Boot-Up Initializer Function:** We introduce a new public static method called **loadPastRecords()** outside of main.
3. **The File Reading Process:** We pass a File object into a new Scanner. The scanner uses a while (fileScanner.hasNextLine()) loop to read every single line of text from the file until it hits the very end.
4. **Zero Code Discarded:** The FileWriter append logic, object structure, and error-handling mechanics remain completely active.

How to Test this Evolution:

1. Compile and run the code. If it's your very first time running it with this updated logic, it will say "No historical database found"
2. Add a student (e.g., "Sam Wilson") and let the program log their grades to disk.
3. Exit the application using choice 3.
4. Run the program a **second time**. Notice what happens instantly on boot-up! The loadPastRecords() block finds the file and outputs Sam's entire profile right back onto your console interface before asking you for new inputs.

Future improvements

- Learn about ArrayLists: Upgrade UnifiedTracker.java so that instead of just flashing reports on screen, the system keeps an active in-memory list tracking every student registered during that loop cycle.
- Log error for duplicate entry.
- Add student member variables for student ID, Second name etc.
- Add UI for user interaction.

```
// — CONTINUOUS MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {

    // — NEW STEP: FUNCTION TO LOAD PAST RECORDS FROM LOCAL DISK —
    public static void loadPastRecords() {
        File databaseFile = new File("report_cards.txt");

        // 1. Check if the backup file actually exists on the computer
        if (!databaseFile.exists()) {
            System.out.println("❗ No historical database found. Starting with a blank registry.\n");
            return; // Exit function early if there is nothing to read
        }

        System.out.println("📁 Loading historical archives from disk drive...");
        System.out.println("=====");

        // 2. Open a specialized scanner locked onto the physical file instead of System.in
        try (Scanner fileScanner = new Scanner(databaseFile)) {

            // 3. Keep loop active as long as there are text strings left to read in the file
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                System.out.println(line); // Output historical file lines directly to console
            }
            System.out.println("=====");
            System.out.println("✅ Data successfully loaded into memory view!\n");

        } catch (IOException e) {
            // Guard against system errors during active read runtime
            System.out.println("❌ Local Storage Failure: Could not safely parse text files.");
        }
    }

    public static void main(String[] args) {
        // — BOOT-UP REUSE: FETCH OLD HISTORY THE MOMENT APPLICATION STARTS —
        loadPastRecords();
    }
}
```